

Universidade do Minho

Algoritmos e Estruturas de Dados

Engenharia de Telecomunicações e Informática

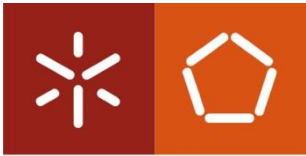
2022/2023

Telmo Adão

Departamento de Sistemas de Informação

Universidade do Minho

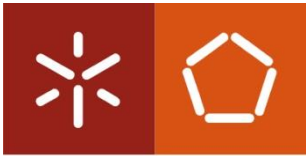
Slides baseados no material didático disponibilizado pelo Professor Luís Magalhães



Universidade do Minho

Sumário

- Continuação da aula anterior (revisões)
 - Revisão de conceitos de C
 - Exercícios



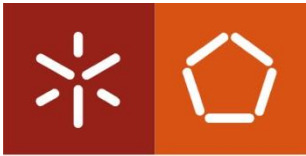
Funções

Universidade do Minho

- Funções - são usadas diretamente em expressões como se fossem simplesmente variáveis.
 - Devolvem sempre um resultado
 - Funções definidas em bibliotecas (ex. `sin()`, `cos()`, `abs()`, `sqr()`, `sqrt()`, ...)
 - Funções definidas pelo utilizador
 - Ex:
 - $\text{Valor} \leftarrow \sin(\text{ângulo}) + \cos(\text{ângulo})$
 - $\text{Valor} \leftarrow \text{sqrt}(\text{variável})$

Em C só existem funções

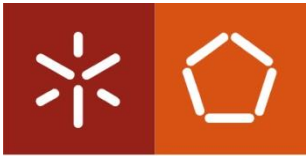
- Os procedimentos são simulados através de funções que não retornam valores



Funções

Universidade do Minho

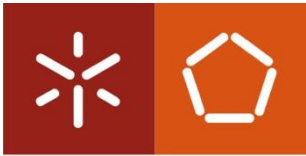
- Formato de definição (continuação)
tipo-valor-retorno nome_função(lista e parâmetros/argumentos)
{
 declarações e instruções...
}
- Definições e instruções: corpo da função(bloco)
 - Variáveis podem ser definidas dentro de blocos
 - Não se pode definir funções dentro de funções
- Controlo de retorno
 - Se retornar “nada” - *void*
 - **return;**
 - ou simplesmente deixa-se correr tudo até à chaveta de fecho do corpo da função
 - Se algo é retornado:
 - **return expressão/variável/valor;**



Exercícios

Universidade do Minho

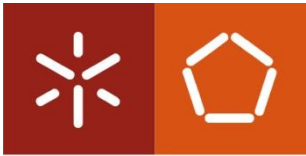
- Uma função que solicite dois números ao utilizador e apresente no ecrã, o resultado da sua soma, e o dobro de cada um deles.
- Uma função que receba como parâmetro um dado inteiro, x , e devolva: -1, se x for negativo; 1 se x for positivo. Um programa que peça x ao utilizador, invoque a rotina em causa e escreva o resultado no ecrã.
- Implemente uma função para escrever no ecrã meia árvore usando asteriscos
 - Criar uma função para escrever n asteriscos no ecrã
 - Utilizar essa função
- Uma função que calcule o fatorial de um dado inteiro, x ; Um programa que peça x ao utilizador, invoque a rotina de cálculo do fatorial e escreva o resultado no ecrã.



Exercícios

Universidade do Minho

- Uma função que receba como parâmetro um dado inteiro, x , calcule e devolva o seu simétrico. Um programa que peça x ao utilizador, evoque a rotina de cálculo do simétrico e escreva o resultado no ecrã.
- Um procedimento que escreva no ecrã, um caracter, seguido dos n caracteres seguintes a esse, na tabela ASCII. Um programa que peça os dados ao utilizador, e os use na evocação da rotina.
- Uma função que calcule o mínimo de cinco números; Uma função que calcule o máximo de cinco números; Uma função que, dados 5 números, calcule a média entre o mínimo e o máximo; Um programa que peça 5 números ao utilizador, e usando aquelas rotinas, apresente no ecrã, a média entre o mínimo e o máximo, dos 5 números introduzidos.



Vetores (arrays)

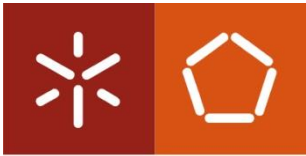
- Vetor (Array)
 - Grupo de consecutivo de “espaços” de memória
 - Mesmo nome e tipo
- Para referir um elemento específico
 - Nome do vetor
 - Número da posição (0-índex based)
- Formato:
 - nome_vetor[posição]
 - O primeiro elemento está na posição 0
 - Elemento **n** do vetor **c**:
 - $c[0], c[1] \dots c[n-1]$

Nome do vetor (todos os elementos do vetor tem o mesmo nome → **c**)



c[0]	-45
c[1]	6
c[2]	0
c[3]	72
c[4]	-89
c[5]	0
c[6]	62
c[7]	-3

Número da posição do elemento dentro do vetor



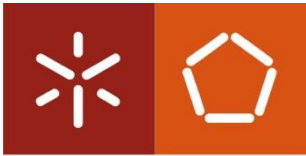
Vetores (arrays)

- Declaração de arrays

tipo nome_variável[nº de elementos]

— Exemplo:

- `int a[10];`
- `float x[7], ab[5];`
- `char b[5];`



Vetores (arrays)

Universidade do Minho

- Inicialização de vetores

```
tipo var[n] = { valor1, valor2, ..., valorn };
```

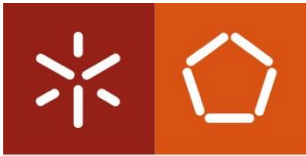
- Exemplo: `int a[5]={2,3,4,7,8};`

Se um vetor for declarado com n elementos e forem colocados apenas k valores ($k < n$) na carga inicial do vetor, então os primeiros k elementos do vetor serão iniciados com os respectivos valores e os restantes serão iniciados com o valor ZERO.

A declaração e a carga inicial de um vetor podem ser realizadas sem indicar qual o número de elementos do vetor.

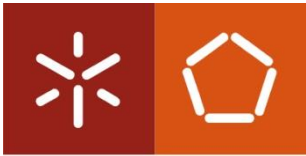
```
tipo var[ ] = { valor1, valor2, ..., valorn };
```

Nesse caso, o compilador vai criar um vetor com tantos elementos quantas as cargas iniciais.

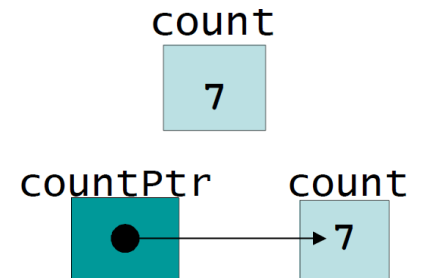


Vetores (arrays)

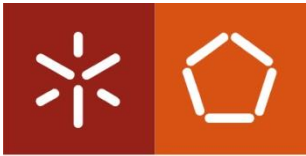
- Exercícios:
 - Dada uma sequência de números (pedida ao utilizador) calcular e apresentar no ecrã, percentagem de números superiores à média.
 - Resolver o exercício anterior criando três funções: preencher o *array*, calcular a média, percentagem de números superiores à media
 - No exercício anterior, acrescentar o cálculo do mínimo.
 - Função que aceite um inteiro e devolva um vetor com os dígitos que o compõem.
 - Analise a complexidade recorrendo à notação O-grande das soluções anteriores



Apontadores

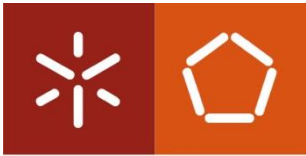


- Pointers ou apontadores
 - Poderosos, mas difíceis de dominar
 - Podem ser usados para simular chamadas por referência
 - Relação íntima com arrays (vetores) e strings (cadeias de caracteres)
- Variáveis apontadoras
 - Guarda um **endereço de memória como valor**
 - Enquanto que as variáveis normais contêm um valor específico (de referência direta), os apontadores contêm um endereço de uma variável de valor específico (referência indireta)
 - Indireção: termo para definir a referência a um valor indicado por meio de apontador



Apontadores

- Definição de pointers, ponteiros ou apontadores
 - * utilizado para designar variáveis deste tipo
`int *myPtr;`
 - No exemplo acima, está definido um apontador de (ou para) inteiro
 - Múltiplas definições requerem a utilização do * antes de cada variável
`int *myPtr, *myPtr1, *myPtr2;`
 - Podem ser definidos apontadores para qualquer tipo de dados
 - Apontadores podem ser inicializados a 0, NULL ou com um endereço de memória
 - 0 ou NULL – apontam para nada (NULL é preferido)



Apontadores

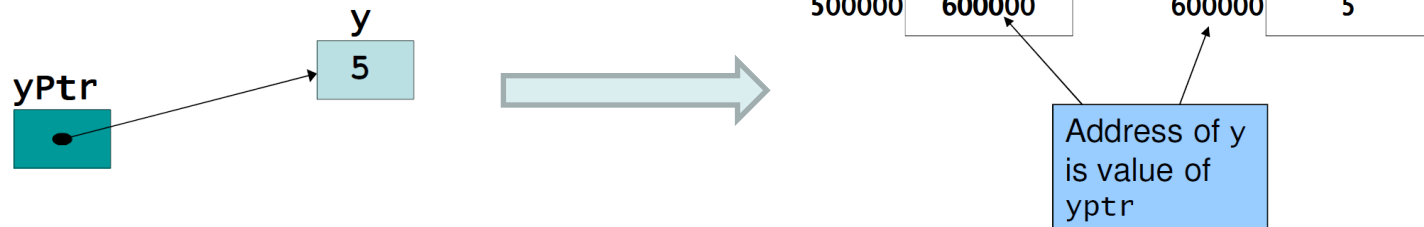
- & (operador de endereço)
 - Retorna o endereço de um operando

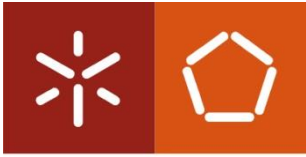
```
Int y = 5;
```

```
Int *yPtr;
```

```
yPtr = &y; /* yPtr recebe o endereço de y */
```

yPtr aponta para y





Universidade do Minho

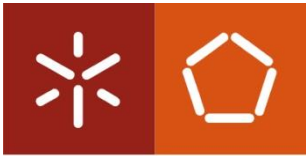
Apontadores

- Passagem de parâmetros nas funções
 - Por valor

Numa passagem de parâmetros por valor, são sempre enviadas para a função cópias dos valores de que esta necessita.

- Por referência
 - Utilização dos apontadores

EM C SÓ EXISTE PASSAGEM DE PARÂMETROS POR VALOR.



Apontadores: passagem por valor

Before main calls cubeByvalue :

```
int main()
{
    int number = 5;
    number=cubeByvalue(number);
}
```

number

5

```
int cubeByvalue( int n )
{
    return n * n * n;
}
```

n

undefined

After cubeByvalue receives the call:

```
int main()
{
    int number = 5;
    number = cubeByvalue( number );
}
```

number

5

```
int cubeByvalue( int n )
{
    return n * n * n;
}
```

n

5

Típica chamada
por valor (1/2)

After cubeByvalue cubes parameter n and before cubeByvalue returns to main:

```
int main()
{
    int number = 5;
    number = cubeByvalue( number );
}
```

number

5

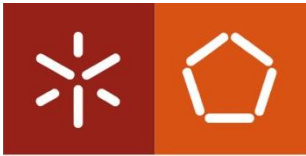
```
int cubeByvalue( int n )
{
    return n * n * n;
}
```

125

n * n * n

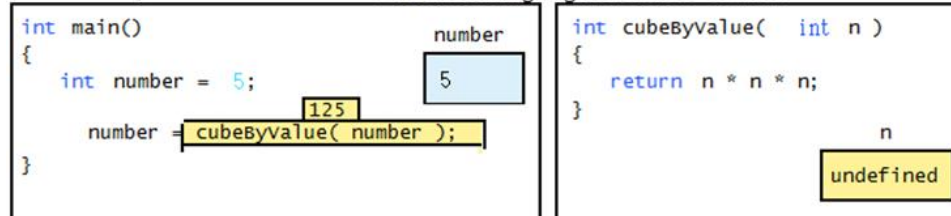
n

5

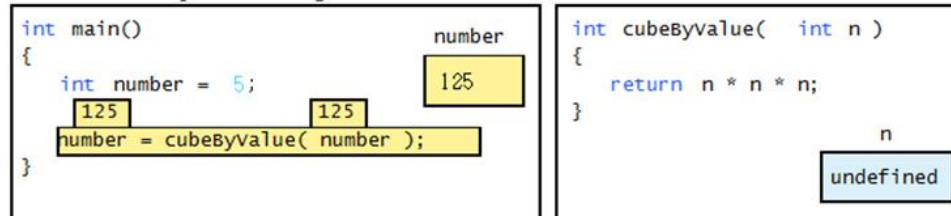


Apontadores: passagem por valor

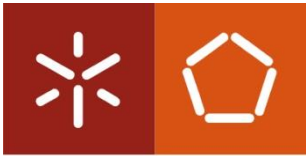
After cubeByvalue returns to main and before assigning the result to number:



After main completes the assignment to number:

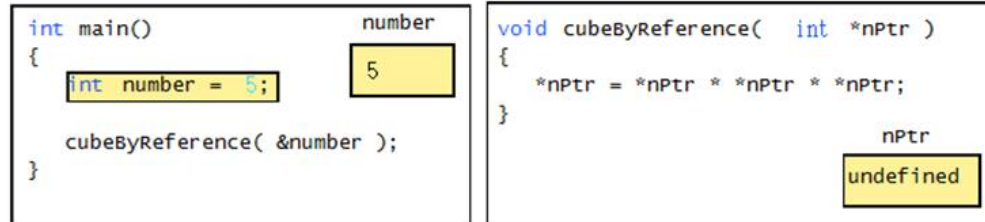


Típica chamada
por valor (2/2)

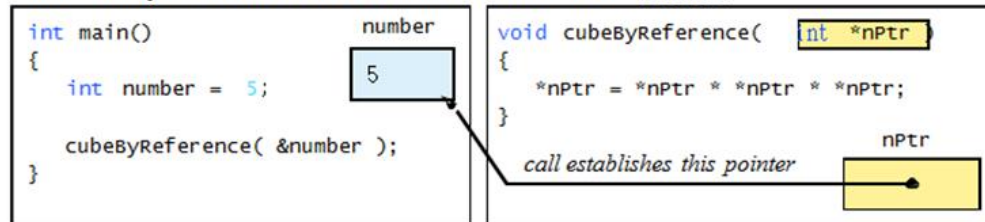


Apontadores: passagem por referência

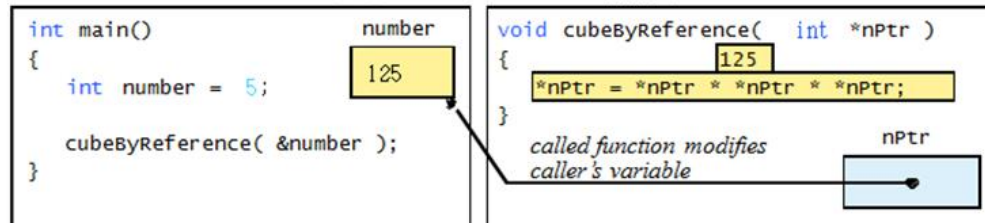
Before main calls cubeByReference :



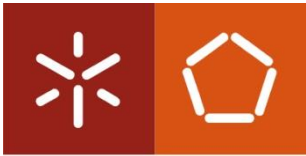
After cubeByReference receives the call and before *nPtr is cubed:



After *nPtr is cubed and before program control returns to main :



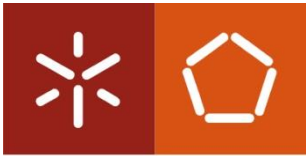
Típica chamada
por referência



Universidade do Minho

Apontadores

- Exercício
 - Construa uma função que permita efetuar a troca dos valores de duas variáveis reais
 - Analise a complexidade recorrendo à notação O-grande das soluções anteriores

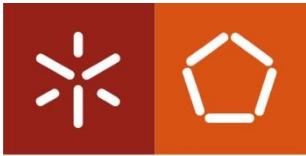


Estruturas ou *Structs*

- Exemplo de definição

```
struct carro {  
    char *marca;  
    char *modelo;  
    int ano_fabrico;  
    float cilindrada;  
    int potencia;  
}Carro;
```

- A palavra *struct* introduz a definição para a estrutura *carro*
- *carro* é o nome da estrutura (define a entidade) e é utilizado para declarar variáveis deste tipo
- *carro* contém 4 membros do tipo *char **, *int* e *float*
 - *marca*, *modelo*, *ano_fabrico*, *cilindrada*, *potencia*



Estruturas ou *Structs*

- Exercício

- Construa uma *struct aluno* com número mecanográfico, nome e nota final.
- Construa uma função para carregar uma variável do tipo *aluno* com nº, nome e nota. A variável é criada no *main* e passada para a função por parâmetro, assim como os dados que a carregam (nº, nome e nota).
- Construa a mesma função, mas desta vez, dentro da própria função, crie a variável complexa *aluno*, carregue-a com os dados nº, nome e nota, que chegam por parâmetro e retorne-a no final. A função deve ser adaptada para suportar devidamente os parâmetros e retorno.